

FACULTAD DE INGENIERÍA, ARQUITECTURA Y URBANISMO PROGRAMA DE
ESTUDIOS DE INGENIERÍA DE SISTEMAS

Guía de producto

“Proyecto de una solución web empresarial.”

Por: Docente del Curso

La siguiente guía tiene por finalidad orientarte en la elaboración del trabajo correspondiente a la evidencia de producto (EP), donde sistematizas la experiencia de trabajo de campo, como parte de una de las actividades obligatorias del curso: Taller de Lenguajes de Programación, por lo tanto, es un documento que debes leerlo minuciosamente, para lograr un trabajo y aprendizaje significativo y alcanzar el mayor puntaje de evaluación.

I. Datos generales

Asignatura	: Taller de Lenguajes de Programación
Ciclo académico	: 2026 – II
Escuela Profesional	: Escuela Académico Profesional de Ingeniería de sistemas
Puntaje de evaluación	: Esta actividad tiene un peso del 40%.

II. Finalidades educativas

A través de esta actividad el estudiante logrará lo siguiente:

Competencia	Capacidades	Dominio de contenidos temáticos	Evidencia de aprendizaje (Producto académico)
Desarrolla software empresarial, utilizando paradigmas y técnicas de programación, demostrando capacidad analítica y promoviendo	Actitudes	Tema 02: Introducción a Django 2.1. Entornos Virtuales 2.2. Que es Django 2.3. Instalación y configuración de Django 2.4. Dockerizando una aplicación de Django Tema 03: ORM Django 3.1. Introducción al ORM de Django 3.2. Configuración de base de datos 3.3. Creación de modelos 3.4. Migraciones 3.4. Relaciones en el ORM 3.5. Django Shell Tema 04: Django Components 4.1. URLs, Views and Templates 4.2. Formularios 4.3. Django Admin Site 4.4. Django Authentication 4.5. Sesiones en Django	Solución web publicada en un repositorio github
	El estudiante será capaz de entender y desarrollar soluciones o web o interfaces programables de aplicaciones (APIs) haciendo uso de Python con Django y Node.js. Además, el estudiante podrá conocer el despliegue de estas soluciones a ambientes productivos.		

el trabajo en equipo		Tema 05: Working with Messages, Email Notifications, and PDF Reports 5.1. Django Messages Framework 5.2. Email notifications 5.3. Custom email templates 5.4. Generando PDFs Tema 06: Django Rest-Framework #1 6.1. Rest in Action 6.2. Function Based Views and Serializers 6.3. Class Based Views 6.4. Mixins 6.5. Generic Views Tema 07: Django Rest-Framework #2 7.1. ViewSets 7.2. Nested Serializers 7.3. Pagination 7.4. Filtering 7.5. Seguridad & Autenticación Tema 08: Tópicos avanzados sobre Django 8.1. Autenticación y Autorización 8.2. Caching, Logging, and Throttling 8.3. Pagination, Django Signals, and Custom Middleware 8.4. Usando Celery con Python y Django	
----------------------	--	--	--

III. Descripción de la actividad

Los estudiantes deberán formar grupos de trabajo de no más de 6 integrantes y deberán desarrollar los siguientes proyectos:

9 Proyectos disponibles

- 1. Plataforma de Delivery Local
- 2. Sistema de Punto de Venta (POS)
- 3. Venta de Pasajes Terrestres
- 4. Optimización de Rutas de Distribución
- 5. Facturación Electrónica (SUNAT-ready)
- 6. Picking inteligente para almacenes
- 7. Sistema de Gestión Hotelera
- 8. Gestión para Restaurantes y Food Service
- 9. Control Nutricional y Tracking de Hábitos

Naturaleza del proyecto: solución web completa

Cada proyecto debe entregarse como una solución web completa que integra backend (Django + DRF) y frontend. No basta con entregar solo la API REST: debe existir una interfaz de usuario funcional y conectada.

Stack tecnológico aceptado (a elegir uno):

- Opción A — Django Templates: vistas renderizadas en servidor con Bootstrap 5 + Alpine.js para interactividad ligera. Recomendado para proyectos con flujos simples de formularios.
- Opción B — SPA Desacoplada: React o Vue.js consumiendo la API REST de Django (DRF). Recomendado para proyectos con mucha interactividad en tiempo real (mapas, KDS, tracking).
- Ambas opciones son válidas. El criterio de evaluación aplica igual para las dos.

Pesos de evaluación globales:

Componente	Peso	Descripción
Modelos y base de datos	15%	Diseño correcto de entidades, relaciones e índices
API REST (endpoints)	15%	Endpoints completos, códigos HTTP correctos, validaciones
Lógica de negocio	15%	Reglas de negocio y flujos críticos implementados
Frontend Web	20%	Vistas funcionales, conectadas a la API, UX usable
Integración API — Frontend	10%	Comunicación correcta, manejo de errores en UI, auth en browser
Autenticación y permisos	10%	JWT/session, roles, acceso protegido en frontend y backend
Testing	10%	Tests unitarios e integración, cobertura $\geq 60\%$
Documentación y calidad	5%	README, Swagger, instrucciones de despliegue, clean code

PROYECTO 1

Plataforma de Delivery Local**Descripción del Sistema**

Plataforma que conecta negocios locales con clientes y repartidores. Permite realizar pedidos en línea, rastrear las entregas en tiempo real y gestionar el ciclo completo, desde la recepción hasta la entrega.

Modelos de Datos Requeridos**Entidades:**

- Negocio: nombre, categoria, direccion, horario (JSON), activo, calificacion_promedio
- Producto: nombre, precio, categoria, stock_disponible, imagen, negocio_id
- Cliente: usuario, direccion_principal, telefono
- Repartidor: usuario, vehiculo, zona_cobertura, disponible, calificacion
- Pedido: cliente, negocio, repartidor(null), estado, subtotal, costo_envio, total, direccion_entrega
- DetallePedido: pedido, producto, cantidad, precio_unitario, subtotal
- SeguimientoEntrega: pedido, latitud, longitud, timestamp

Estados del pedido: RECIBIDO → CONFIRMADO → EN_PREPARACION → LISTO_PARA_RECOJO → EN_CAMINO → ENTREGADO / CANCELADO

Endpoints Mínimos Requeridos

- POST /api/pedidos/ — Crear pedido con validación de stock y horario del negocio
- GET /api/pedidos/{id}/tracking/ — Estado actual + historial de movimientos
- PATCH /api/pedidos/{id}/estado/ — Cambiar estado con validación transición
- GET /api/negocios/?categoria=&disponible=true — Listar negocios con filtros
- GET /api/negocios/{id}/productos/ — Catálogo del negocio
- POST /api/repartidores/{id}/ubicacion/ — Actualizar posición GPS
- GET /api/clientes/mis-pedidos/ — Historial del cliente autenticado
- POST /api/pedidos/{id}/calificar/ — Calificar entrega (1-5 estrellas)

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Home / Landing: listado de negocios con buscador y filtros por categoria. Cada negocio muestra nombre, horario, calificación y estado (abierto/cerrado)
- Detalle Negocio: Catálogo de productos con imagen, precio y boton "Agregar al carrito". Indicador de stock disponible
- Carrito de Compras: resumen de items, subtotal, costo de envio y total. Campo para ingresar direccion de entrega y boton de confirmar pedido
- Mis Pedidos: listado de pedidos del cliente con estado actual (badge con color), fecha y total. Botón para ver tracking
- Tracking del Pedido: timeline visual de los estados del pedido con timestamps. Indicador del repartidor asignado si aplica
- Panel Negocio — Pedidos Entrantes: tabla en tiempo real de pedidos nuevos con botón para confirmar o rechazar cada uno. Cambio de estado por pedido
- Panel Repartidor — Mi Pedido Activo: detalle del pedido asignado con dirección de entrega y botones para actualizar estado (Recogido, Entregado)
- Panel Admin — Dashboard: resumen de pedidos del dia, negocios activos, repartidores disponibles

Reglas de Negocio Críticas

- No se puede crear un pedido si el negocio está cerrado según su horario
- No se puede pedir un producto con stock = 0
- Solo el repartidor asignado puede marcar EN_CAMINO o ENTREGADO
- El cliente solo puede cancelar en estado RECIBIDO o CONFIRMADO
- Un repartidor solo puede tener un pedido EN_CAMINO a la vez

- La calificación es única por pedido entregado

Criterios de aceptación

1. API: crear pedido, cambiar estado, tracking, calificación — todas con validaciones
2. Frontend cliente: home, carrito, confirmación y mis pedidos navegables y funcionales
3. Frontend negocio: pedidos entrantes actualizados, cambio de estado operativo
4. Frontend repartidor: pedido activo con actualización de estado
5. Auth: login/registro con 3 roles, vistas protegidas por rol, redirect si no autenticado

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades con relaciones e índices definidos	Entidades principales, faltan índices o constraints	Modelos básicos con relaciones incompletas	Modelos incompletos
API REST 15%	15%	Todos los endpoints con validaciones, códigos HTTP correctos y paginación	Endpoints principales funcionando con validaciones parciales	CRUD básico sin validaciones de negocio	Menos del 50% de endpoints
Maquina de estados 15%	15%	Transiciones completas con validación y error descriptivo	Estados implementados con algunas transiciones faltantes	Cambio de estado sin validación de transiciones	Sin maquina de estados
Frontend Web 20%	20%	Todas las vistas funcionales, conectadas a la API, UX clara con estados de carga y mensajes de error	Vistas principales funcionales, algunas sin manejo de errores o sin feedback visual	Vistas básicas con formularios funcionando pero sin feedback ni estilos coherentes	Frontend inexistente o no conectado a la API
Integración API-Frontend 10%	10%	Token JWT almacenado y enviado en headers, errores de API mostrados en UI, redirect al expirar sesión	Token manejado pero algunos endpoints sin manejo de error en frontend	Llamadas a la API funcionan pero sin manejo de errores o autenticación en browser	Sin integración funcional
Auth y Permisos 10%	10%	JWT con 3 roles, vistas protegidas, datos filtrados por rol en frontend y backend	JWT con roles pero vistas no protegidas o datos sin filtrar	Autenticación básica sin roles	Sin autenticación
Testing 10%	10%	Tests de flujo de pedido, maquina de estados y permisos. >= 60% cobertura	Tests principales 40-60%	Tests básicos < 40%	Sin tests

Docs y calidad 5%	5%	Swagger, README con instrucciones de instalacion frontend y backend, sin N+1	Swagger y README basico	Solo README	Sin documentacion
----------------------	----	--	-------------------------	-------------	-------------------

PROYECTO 2

Sistema de Punto de Venta (POS)

Descripcion del Sistema

Sistema para gestionar las ventas en el mostrador, el control de inventario, la apertura y el cierre de caja, y los reportes de ventas diarios y por producto. Orientado a negocios retail.

Modelos de Datos Requeridos

Entidades:

- Producto: codigo_barra, nombre, categoria, precio_venta, costo, stock_actual, stock_minimo, unidad
- Proveedor: nombre, ruc, contacto
- Venta: cajero, caja, fecha, subtotal, descuento, igv, total, metodo_pago, estado
- DetalleVenta: venta, producto, cantidad, precio_unitario, descuento_linea, subtotal
- Caja: nombre, saldo_inicial, saldo_final, fecha_apertura, fecha_cierre, cajero, estado
- MovimientoCaja: caja, tipo(INGRESO/EGRESO), monto, concepto
- Compra: proveedor, fecha, total, estado
- DetalleCompra: compra, producto, cantidad, costo_unitario

Endpoints Minimos Requeridos

- POST /api/ventas/ — Registrar venta, descontar stock, actualizar saldo de caja
- POST /api/cajas/{id}/apertura/ — Abrir caja con saldo inicial
- POST /api/cajas/{id}/cierre/ — Cerrar caja y calcular diferencia
- GET /api/productos/buscar/?q= — Búsqueda por nombre o código de barras
- GET /api/reportes/ventas-del-dia/ — Total, por método de pago, top productos
- GET /api/reportes/stock-critico/ — Productos bajo el mínimo
- POST /api/compras/ — Registrar entrada de mercadería e incrementar stock
- GET /api/reportes/utilidad/?periodo= — Ingresos vs costos por periodo

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Pantalla de Venta (Caja): buscador de producto por nombre o código de barra, lista de items del ticket en curso con cantidad editable, subtotales y total. Botón "Cobrar" con selector de método de pago
- Apertura de Caja: formulario para ingresar saldo inicial, historial de apertura/cierre del día
- Cierre de Caja: resumen automático de ingresos y egresos del turno, diferencia de caja, botón confirmar cierre
- Inventario: tabla de productos con búsqueda, filtro por categoría, indicador de stock crítico (fila resaltada en rojo), botones CRUD
- Entrada de Mercadería: formulario de compra a proveedor con selección de productos y cantidades. Actualización de stock automática al guardar
- Reportes — Ventas del Día: totales por método de pago, gráfico de top 5 productos, listado de ventas con filtro por hora
- Reportes — Stock Crítico: tabla de productos bajo el mínimo con columna de sugerencia de reposición
- Panel Admin — Dashboard: ventas del día, caja actual, alertas de stock, accesos directos a reportes

Reglas de Negocio Críticas

- No se puede vender si la caja no está ABIERTA
- No se puede vender un producto con stock = 0
- El saldo de cierre = saldo_inicial + ingresos - egresos
- Las ventas no se eliminan, solo se anulan con motivo obligatorio
- precio_venta no puede ser menor al costo (validación de margen mínimo)

Criterios de aceptación

6. Pantalla de venta: agregar productos, calcular totales, cobrar y generar ticket
7. Caja: apertura, cierre y cuadro automático funcionando
8. Inventario: CRUD completo con alertas de stock crítico visibles
9. Reportes: ventas del día y utilidad con filtros de fecha operativos
10. Auth: cajero vs admin, cajero no puede acceder a módulo de compras ni reportes de utilidad

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades con constraints de stock >= 0 e índices en código_barra	Entidades principales sin constraints de stock	Modelos básicos sin índices de rendimiento	Modelos incompletos

API REST 15%	15%	Todos los endpoints con actualizacion atomica de stock y caja	Endpoints principales, actualizacion de stock sin atomicidad	CRUD basico sin logica de caja ni stock	Menos del 50%
Logica de caja 15%	15%	Apertura, cierre, movimientos, cuadre automatico y validación de caja abierta	Apertura y cierre sin validación de estado en ventas	Logica de caja parcial o calculos incorrectos	Sin modulo de caja
Frontend Web 20%	20%	Pantalla de venta interactiva con buscador, ticket en tiempo real y cobro. Reportes con graficas	Pantalla de venta y caja funcionales, reportes solo en tabla sin graficas	Formularios basicos sin feedback visual ni UX de caja	Frontend inexistente o desconectado
Integracion API-Frontend 10%	10%	Busqueda en tiempo real, actualizacion de stock visible al cerrar venta, errores mostrados en UI	Flujo principal integrado con algunos errores sin capturar en UI	Llamadas a la API funcionan pero sin UX de retroalimentacion	Sin integracion funcional
Auth y Permisos 10%	10%	JWT con roles (admin, cajero, supervisor), vistas protegidas y menu diferenciado por rol	JWT con roles pero vistas accesibles sin restriccion	Autenticacion basica sin roles	Sin autenticacion
Testing 10%	10%	Tests de flujo de venta, apertura/cierre y stock. >= 60%	Tests principales 40-60%	Tests basicos < 40%	Sin tests
Docs y calidad 5%	5%	Swagger, README con instrucciones de setup, sin N+1	Swagger y README basico	Solo README	Sin documentacion

PROYECTO 3

Venta de Pasajes Terrestres Interprovinciales

Descripcion del Sistema

Plataforma para la búsqueda, reserva y compra de pasajes interprovinciales. Gestiona múltiples empresas de transporte, rutas, horarios y disponibilidad de asientos en tiempo real.

Modelos de Datos Requeridos

Entidades:

- EmpresaTransporte: nombre, ruc, logo, activo
- Terminal: nombre, ciudad, direccion, empresa
- TipoBus: nombre, num_asientos, configuracion_asientos (JSON), empresa
- Ruta: origen(Terminal), destino(Terminal), distancia_km, duracion_estimada
- Viaje: ruta, empresa, tipo_bus, fecha_salida, hora_salida, precio_base, estado
- Asiento: viaje, numero, fila, columna, tipo(NORMAL/VIP/CAMA), estado(DISPONIBLE/RESERVADO/VENDIDO)
- Reserva: viaje, pasajero, asiento, estado, expiracion (TTL 15 min), codigo_reserva
- Boleto: reserva, nombres, apellidos, dni, precio_final, serie, numero, estado

Endpoints Minimos Requeridos

- GET /api/viajes/buscar/?origen=&destino=&fecha= — Disponibilidad con asientos
- GET /api/viajes/{id}/asientos/ — Mapa de asientos con estado en tiempo real
- POST /api/reservas/ — Bloquear asiento por 15 minutos (TTL)
- POST /api/boletos/ — Confirmar compra atómica
- GET /api/boletos/{codigo}/ — Detalle público del boleto
- POST /api/boletos/{codigo}/cancelar/ — Cancelación con política de reembolso
- GET /api/empresas/{id}/viajes/?fecha= — Programación de viajes
- GET /api/reportes/ocupacion/?viaje_id= — Porcentaje de ocupación

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Buscador de Viajes: formulario con autocomplete de origen y destino, selector de fecha, numero de pasajeros. Boton buscar prominente
- Resultados de Búsqueda: lista de viajes disponibles con empresa, horario, tipo de bus, precio y asientos disponibles. Filtros por empresa y tipo de asiento
- Selección de Asiento: mapa visual e interactivo del bus con asientos coloreados por estado (verde=libre, rojo=ocupado, amarillo=reservado). Click para seleccionar
- Checkout: formulario con datos del pasajero (nombre, apellido, DNI), resumen del viaje y asiento seleccionado, total a pagar y boton confirmar. Contador regresivo de 15 min del TTL
- Mi Boleto: vista del comprobante con datos del viaje, asiento, pasajero y codigo QR (o codigo alfanumerico). Boton de cancelar si aplica
- Panel Empresa — Viajes: listado de viajes del dia con boton de ver ocupacion. Porcentaje de ocupacion por viaje en barra de progreso
- Panel Empresa — Ocupacion de Viaje: mapa del bus mostrando asientos vendidos y libres con nombre del pasajero en hover
- Consulta Publica de Boleto: pagina sin login para verificar validez del boleto por codigo

Reglas de Negocio Críticas

- El asiento reservado se libera automáticamente a los 15 min si no se confirma la compra
- No se puede reservar un asiento RESERVADO o VENDIDO (manejo de concurrencia)
- Un pasajero no puede comprar dos asientos en el mismo viaje con el mismo DNI
- Cancelación > 24 h antes: reembolso del 80%. Menos de 24 h: sin reembolso

Criterios de aceptación

11. Búsqueda y resultados: filtro correcto con disponibilidad real de asientos
12. Mapa de asientos: colores por estado, selección interactiva y actualización al reservar
13. Flujo de compra: TTL visible, confirmación atómica y boleto generado
14. Mi boleto: consulta pública por código, opción de cancelar con política aplicada
15. Panel empresa: ocupación por viaje visible y actualizada

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades con unique en asiento+viaje y constraints de estado	Entidades principales sin unique together	Modelos basicos sin control de concurrencia	Modelos incompletos
API REST 15%	15%	Busqueda, reserva TTL y compra atomica con manejo de concurrencia	Flujo principal sin TTL o sin control de concurrencia	Busqueda y compra sin validaciones de conflicto	Menos del 50%
Logica de reserva TTL 15%	15%	Bloqueo temporal 15 min, liberacion automatica, compra atomica con rollback	Bloqueo implementado sin liberacion automatica	Reserva sin TTL ni atomicidad	Sin logica de reserva temporal
Frontend Web 20%	20%	Mapa interactivo de asientos, contador TTL visible, flujo completo de compra y consulta de boleto	Mapa de asientos y flujo de compra sin contador TTL o sin mapa interactivo	Formularios de busqueda y compra sin mapa de asientos	Frontend inexistente o desconectado
Integracion API-Frontend 10%	10%	Mapa se actualiza en tiempo real, TTL cuenta regresiva en browser, errores de disponibilidad mostrados	Flujo integrado sin actualizacion en tiempo real del mapa	Llamadas a la API funcionan sin feedback de disponibilidad	Sin integracion funcional
Auth y Permisos 10%	10%	JWT con roles (pasajero, empresa, admin), datos filtrados por empresa en panel	JWT con roles parciales	Autenticacion basica	Sin autenticacion

Testing 10%	10%	Tests de búsqueda, reserva concurrente, TTL y cancelacion. >= 60%	Tests del flujo principal 40-60%	Tests basicos < 40%	Sin tests
Docs y calidad 5%	5%	Swagger, README con instrucciones de setup completo	Swagger y README basico	Solo README	Sin documentacion

PROYECTO 4

Optimizacion de Rutas de Distribucion

Descripcion del Sistema

Sistema para planificar y optimizar rutas de entrega. Asigna pedidos a vehículos según capacidad, zona y prioridad, registra el avance y genera KPIs de eficiencia logística.

Modelos de Datos Requeridos

Entidades:

- Vehiculo: placa, tipo, capacidad_kg, capacidad_bultos, conductor, disponible
- Conductor: usuario, licencia, celular, activo
- PuntoEntrega: cliente_nombre, direccion, zona, latitud, longitud, telefono
- PedidoDespacho: codigo, punto_entrega, peso_kg, bultos, prioridad, ventana_horaria, estado
- Ruta: vehiculo, fecha, estado, hora_inicio, hora_fin, km_estimados, km_reales
- ParadaRuta: ruta, pedido, orden, hora_estimada, hora_real, estado, observacion
- IndicadorRuta: ruta, pedidos_asignados, pedidos_entregados, pedidos_fallidos, km, tiempo_min

Endpoints Minimos Requeridos

- POST /api/rutas/planificar/ — Crear ruta asignando pedidos a vehículo por zona y capacidad
- GET /api/rutas/{id}/paradas/ — Secuencia de paradas con horas estimadas
- PATCH /api/paradas/{id}/estado/ — Actualizar estado (ENTREGADO, FALLIDO, REPROGRAMADO)
- GET /api/rutas/en-curso/ — Rutas activas con posición actual
- POST /api/vehiculos/{id}/posicion/ — Actualizar posición GPS
- GET /api/reportes/eficiencia/?fecha= — KPIs: % entrega, km, pedidos por vehículo
- GET /api/pedidos/sin-asignar/ — Pedidos pendientes
- GET /api/zonas/{id}/pedidos-hoy/ — Pedidos del día por zona

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Dashboard Supervisor — Mapa del día: mapa con iconos de vehículos en ruta, colores según estado (en camino/completada/con incidencia). Panel lateral con resumen de rutas del día
- Planificación de Ruta: listado de pedidos sin asignar con peso y zona. Formulario para seleccionar vehículo y pedidos a incluir. Validación visual de capacidad (barra de progreso de peso)
- Detalle de Ruta: secuencia de paradas con nombre del cliente, dirección, hora estimada y estado actual (PENDIENTE/EN_CAMINO/ENTREGADO/FALLIDO). Timeline vertical
- Panel Conductor — Mi Ruta: vista móvil-friendly de la parada actual con nombre, dirección y referencia. Botones grandes para marcar ENTREGADO o FALLIDO con campo de observación
- Pedidos Sin Asignar: tabla con filtros por zona, prioridad y fecha. Botón "Planificar ruta" para los seleccionados
- Reportes de Eficiencia: KPIs por fecha — porcentaje de entrega, km recorridos, tiempo promedio. Comparativa por vehículo en tabla y gráfico de barras
- Historial de Rutas: listado de rutas cerradas con sus indicadores. Click para ver el detalle de paradas

Reglas de Negocio Críticas

- Un vehículo no puede tener dos rutas activas al mismo tiempo
- La suma de pesos en una ruta no puede superar la capacidad del vehículo
- Los pedidos URGENTES tienen prioridad en el ordenamiento de paradas
- La ruta no puede cerrarse si tiene paradas pendientes
- Las ventanas horarias deben respetarse en el orden de paradas

Criterios de aceptación

- Planificación: asignación por zona y capacidad con validación de restricciones visible
- Mapa: vehículos visibles con estado actualizado en tiempo real
- Conductor: panel de parada actual con botones de confirmación funcionando
- Cierre de ruta: KPIs calculados y guardados automáticamente
- Reportes: eficiencia con filtros por fecha y vehículo

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades, constraint de capacidad, secuencia como JSON	Entidades principales sin constraint de capacidad	Modelos básicos sin trazabilidad de paradas	Modelos incompletos

API REST 15%	15%	Planificación, tracking y cierre con todas las validaciones	Flujo principal sin validaciones de capacidad o prioridad	CRUD básico sin lógica de planificación	Menos del 50%
Lógica de planificación 15%	15%	Asignación por zona y capacidad, ordenamiento por prioridad y ventana horaria	Asignación sin ordenamiento por prioridad	Asignación manual sin validaciones	Sin lógica de planificación
Frontend Web 20%	20%	Mapa con vehículos, dashboard de rutas, panel conductor y reportes con gráficas funcionando	Mapa y dashboard sin panel conductor o sin gráficas en reportes	Solo tablas sin mapa ni feedback visual de planificación	Frontend inexistente o desconectado
Integración API-Frontend 10%	10%	Mapa actualiza posición, estado de paradas se refleja en UI, validación de capacidad visible al planificar	Flujo integrado sin actualización dinámica del mapa	Llamadas funcionan sin feedback de restricciones	Sin integración funcional
Auth y Permisos 10%	10%	JWT con roles (supervisor, conductor, admin), conductor solo ve sus rutas	JWT con roles parciales	Autenticación básica	Sin autenticación
Testing 10%	10%	Tests de planificación, validación de capacidad y cierre. $\geq 60\%$	Tests principales 40-60%	Tests básicos $< 40\%$	Sin tests
Docs y calidad 5%	5%	Swagger, README, sin N+1 en rutas con paradas	Swagger y README básico	Solo README	Sin documentación

PROYECTO 5

Sistema de Facturación Electrónica SUNAT-ready

Descripción del Sistema

Sistema para emisión de comprobantes electrónicos (facturas, boletas, notas de crédito) siguiendo la normativa de la SUNAT. Gestiona el ciclo completo: emisión, envío al OSE (mock) y respuesta de aceptación.

Modelos de Datos Requeridos

Entidades:

- Empresa: ruc, razon_social, nombre_comercial, direccion, regimen_tributario
- SerieComprobante: tipo(F/B/FC), serie, correlativo_actual, empresa
- Cliente: tipo_doc(RUC/DNI/CE), num_doc, razon_social, direccion, email
- Producto: codigo, descripcion, unidad_medida, precio_unitario, afecto_igv
- Comprobante: serie, numero, fecha, cliente, tipo, subtotal, igv, total, estado, xml_firmado
- DetalleComprobante: comprobante, producto, cantidad, precio_unitario, descuento, igv_linea, subtotal
- LogEnvioSUNAT: comprobante, fecha_envio, estado_respuesta, codigo_respuesta, descripcion
- NotaCredito: comprobante_referencia, motivo, tipo_nota, monto_afectado

Endpoints Minimos Requeridos

- POST /api/facturas/ — Emitir factura con cálculo de IGV y generación de XML (mock firma)
- POST /api/boletas/ — Emitir boleta de venta
- POST /api/notas-credito/ — Emitir nota de crédito referenciando el original
- GET /api/comprobantes/?tipo=&fecha_desde=&ruc_cliente= — Listado con filtros
- POST /api/comprobantes/{id}/reenviar/ — Reenviar si estado es RECHAZADO
- GET /api/comprobantes/{id}/pdf/ — Vista imprimible del comprobante
- GET /api/reportes/ventas-por-periodo/?mes=&anio= — Libro de ventas simplificado

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Dashboard: resumen del mes — total facturas emitidas, total boletas, monto total, alertas de comprobantes RECHAZADOS pendientes de reenvio
- Emitir Comprobante: formulario con selector de tipo (Factura/Boleta), buscador de cliente (por RUC o DNI con autocompletado), tabla de líneas de productos con calculo automatico de IGV y totales en tiempo real
- Lista de Comprobantes: tabla paginada con filtros por tipo, fecha, cliente y estado SUNAT. Cada fila muestra badge de estado (ACEPTADO en verde, RECHAZADO en rojo, ENVIADO en amarillo)
- Vista Previa del Comprobante: representacion visual del comprobante estilo voucher con datos de la empresa, cliente, líneas de detalle, subtotal, IGV y total. Boton de imprimir o descargar PDF
- Emitir Nota de Credito: formulario que busca el comprobante original por serie/numero, muestra sus datos y permite seleccionar el motivo y monto a anular
- Mantenimiento de Clientes: CRUD con busqueda por RUC o DNI, detalle de comprobantes emitidos por cliente
- Mantenimiento de Productos/Servicios: CRUD con precio, unidad y flag de afecto a IGV
- Reporte Libro de Ventas: tabla del mes con todos los comprobantes, totales de base imponible, IGV y total. Exportar a Excel (o CSV)

Reglas de Negocio Críticas

Material didáctico de apoyo al estudiante

- La numeración es correlativa y única por serie, sin saltos
- La factura requiere RUC (11 dígitos). Boleta acepta DNI
- IGV = 18% sobre la base imponible de productos afectos
- Nota de crédito no puede superar el monto del original
- Comprobante ACEPTADO no se puede eliminar, solo anular vía nota de crédito
- Estado sigue: BORRADOR → EMITIDO → ENVIADO → ACEPTADO / RECHAZADO

Criterios de aceptación

21. Emisión: cálculo correcto de IGV y totales, numeración correlativa automática
22. UI de emisión: autocompletado de cliente, líneas dinámicas, total en tiempo real
23. Vista del comprobante: formato voucher con todos los datos y botón de imprimir
24. Nota de crédito: referencia al original, validación de monto, actualización de estado
25. Reportes: libro de ventas por mes con totales correctos

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades, unique en serie+numero, log de envios SUNAT	Entidades principales sin unique o sin log de envios	Modelos basicos sin auditoria de SUNAT	Modelos incompletos
API REST 15%	15%	Emision de los 3 tipos con validaciones tributarias, reenvio y ciclo de estados	Factura y boleta, nota de credito parcial	Solo facturas sin validaciones tributarias	Menos del 50%
Logica tributaria 15%	15%	IGV correcto, numeracion correlativa, ciclo SUNAT implementado con estados	IGV correcto pero numeracion o estados con errores	Calculo basico sin control de numeracion	Sin logica tributaria
Frontend Web 20%	20%	Formulario con autocompletado, calculo en tiempo real, vista previa de comprobante, lista con badges de estado	Formulario funcional y lista de comprobantes sin vista previa ni autocompletado	Formularios basicos sin calculo dinamico ni vista del comprobante	Frontend inexistente o desconectado
Integracion API-Frontend 10%	10%	Totales calculados dinamicamente en browser, estado SUNAT actualizado visualmente, errores tributarios mostrados en UI	Flujo integrado sin calculo dinamico de totales	Llamadas funcionan sin feedback tributario	Sin integracion funcional
Auth y Permisos 10%	10%	JWT con roles (emisor, contador, admin), comprobantes filtrados por empresa	JWT con roles parciales	Autenticacion basica	Sin autenticacion

Testing 10%	10%	Tests de calculo tributario, numeracion y ciclo SUNAT. >= 60%	Tests de emision basica 40-60%	Tests basicos < 40%	Sin tests
Docs y calidad 5%	5%	Swagger, README con detalle de estructura XML usada y setup	Swagger y README basico	Solo README	Sin documentacion

PROYECTO 6

Sistema de Picking Inteligente para Almacenes

Descripcion del Sistema

Sistema para gestionar órdenes de picking: recibe pedidos, genera tareas para los operarios, optimiza la ruta dentro del almacén y controla los tiempos y la productividad.

Modelos de Datos Requeridos

Entidades:

- Almacen: nombre, codigo, num_pasillos, num_niveles
- Ubicacion: almacen, pasillo, estante, nivel, codigo_ubicacion
- Producto: sku, descripcion, unidad, peso_kg, categoria, ubicacion_principal
- StockUbicacion: ubicacion, producto, cantidad, lote
- OrdenPedido: numero, cliente, prioridad, fecha_solicitud, fecha_requerida, estado
- LineaPedido: orden, producto, cantidad_solicitada, cantidad_pickeada, estado
- TareaPicking: orden, operario, estado, fecha_asignacion, fecha_inicio, fecha_fin, secuencia_paradas (JSON)
- PickingDetalle: tarea, linea_pedido, ubicacion, cantidad_solicitada, cantidad_real, conforme, observacion
- IndicadorOperario: operario, fecha, ordenes_completadas, lineas_pickeadas, tiempo_promedio_min, precision_pct

Endpoints Minimos Requeridos

- POST /api/tareas/generar/{orden_id}/ — Crear tarea optimizando secuencia de ubicaciones
- GET /api/tareas/{id}/secuencia/ — Lista ordenada de ubicaciones a visitar
- POST /api/tareas/{id}/iniciar/ — Registrar hora de inicio
- POST /api/picking-detalles/{id}/confirmar/ — Confirmar cantidad pickeada
- POST /api/tareas/{id}/completar/ — Cerrar tarea y calcular KPIs
- GET /api/operarios/{id}/productividad/?fecha= — KPIs del operario
- GET /api/stock/buscar/?sku= — Ubicaciones con stock del producto
- GET /api/reportes/picking/?fecha_desde=&fecha_hasta= — Eficiencia del almacén

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Dashboard Supervisor: ordenes pendientes de asignacion, operarios disponibles, tareas en curso y completadas del dia. Indicadores globales: ordenes en tiempo, atrasadas, precision del dia
- Asignacion de Tarea: tabla de ordenes pendientes con columnas de prioridad, peso total y fecha requerida. Selector de operario disponible. Vista previa de la secuencia de paradas al asignar
- Vista Operario — Mi Tarea Actual: interfaz simple tipo "paso a paso" con la parada actual resaltada: ubicacion (pasillo-estante-nivel), producto y cantidad solicitada. Barra de progreso de paradas completadas
- Vista Operario — Confirmar Picking: formulario por parada para ingresar cantidad real encontrada. Si difiere de lo solicitado, campo de observacion obligatorio. Boton de siguiente parada
- Vista Almacen — Mapa de Ubicaciones: grid o tabla que representa los pasillos y estantes. Colores segun nivel de stock (verde=ok, amarillo=bajo, rojo=agotado)
- KPIs de Operario: tabla de productividad por operario por dia — ordenes completadas, precision, tiempo promedio. Grafico de tendencia semanal por operario
- Historial de Ordenes: listado con filtros por estado, fecha y operario. Click para ver el detalle de paradas y discrepancias encontradas

Reglas de Negocio Críticas

- Una tarea solo puede asignarse a un operario sin tarea activa
- La secuencia debe minimizar el recorrido (ordenar por pasillo y estante)
- Discrepancia en cantidad requiere observación obligatoria
- El stock se descuenta al confirmar cada picking
- Una tarea no puede completarse si tiene detalles pendientes

Criterios de aceptación

26. Generación de tarea: asignación de operario y secuencia optimizada visible
27. UI operario: flujo paso a paso claro, confirmación de cantidad con validación
28. Descuento de stock: visible en mapa de almacén después de confirmar picking
29. KPIs: productividad por operario calculada y mostrada en gráfico
30. Historial: órdenes con detalle de discrepancias accesible

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
---------------	------	----------------	-------------------	--------------	-------------------

Modelos y DB 15%	15%	Todas las entidades con stock transaccional y secuencia como JSON	Entidades principales sin atomicidad en stock	Modelos basicos sin trazabilidad de picking	Modelos incompletos
API REST 15%	15%	Generacion, confirmacion atomica y cierre con KPIs calculados	Flujo principal con KPIs parciales	CRUD basico sin logica de picking	Menos del 50%
Logica de optimizacion 15%	15%	Secuencia optimizada por pasillo/estante, asignacion de operario disponible	Secuencia sin criterio de optimizacion	Asignacion manual sin optimizacion	Sin logica de optimizacion
Frontend Web 20%	20%	Dashboard, UI paso a paso del operario, mapa de almacen y KPIs con graficas funcionando	Dashboard y UI operario sin mapa de almacen ni graficas	Solo tablas sin flujo guiado para el operario	Frontend inexistente o desconectado
Integracion API-Frontend 10%	10%	Stock del mapa se actualiza al confirmar, KPIs recalculan al completar tarea, progreso visible	Flujo integrado sin actualizacion dinamica del stock	Llamadas funcionan sin feedback de progreso	Sin integracion
Auth y Permisos 10%	10%	JWT con roles (operario, supervisor, admin), operario solo ve sus tareas	JWT con roles parciales	Autenticacion basica	Sin autenticacion
Testing 10%	10%	Tests de generacion, picking y cierre. >= 60%	Tests principales 40-60%	Tests basicos < 40%	Sin tests
Docs y calidad 5%	5%	Swagger, README, sin N+1 en tareas con detalles	Swagger y README basico	Solo README	Sin documentacion

PROYECTO 7

Sistema de Gestion Hotelera

Descripcion del Sistema

Plataforma para administrar reservas, check-in/check-out, housekeeping, facturación y ocupación de habitaciones. Gestiona tarifas dinámicas por temporada y por tipo de habitación.

Modelos de Datos Requeridos

Entidades:

- Hotel: nombre, ruc, direccion, estrellas, telefono
- TipoHabitacion: nombre, capacidad, precio_base, amenidades (JSON)
- Habitacion: hotel, tipo, numero, piso, estado(DISPONIBLE/OCUPADA/LIMPIEZA/MANTENIMIENTO)
- Huesped: tipo_doc, num_doc, nombres, apellidos, email, telefono, nacionalidad
- Tarifa: tipo_habitacion, nombre, precio_noche, fecha_inicio, fecha_fin
- Reserva: hotel, huesped, habitacion, fecha_entrada, fecha_salida, num_adultos, estado, precio_total, origen
- Estancia: reserva, habitacion, fecha_checkin, fecha_checkout, precio_final, estado
- CargoEstancia: estancia, concepto, monto, fecha, tipo
- Folio: estancia, subtotal, igv, total, estado

Endpoints Minimos Requeridos

- GET /api/habitaciones/disponibles/?fecha_entrada=&fecha_salida=&tipo= — Disponibilidad
- POST /api/reservas/ — Crear reserva con cálculo de precio según tarifa vigente
- POST /api/reservas/{id}/checkin/ — Check-in con asignación de habitación
- POST /api/estancias/{id}/checkout/ — Check-out y cálculo de folio final
- POST /api/estancias/{id}/cargos/ — Agregar cargo adicional
- GET /api/estancias/{id}/folio/ — Ver folio con todos los cargos
- PATCH /api/habitaciones/{id}/housekeeping/ — Actualizar estado de limpieza
- GET /api/reportes/ocupacion/?fecha= — Tasa de ocupación del día

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Plano del Hotel (Mapa de Habitaciones): grid visual con cada habitacion representada como una celda coloreada segun su estado (verde=disponible, rojo=ocupada, amarillo=limpieza, gris=mantenimiento). Click para ver detalle
- Calendario de Reservas: vista tipo timeline horizontal por habitacion mostrando reservas como bloques de colores con nombre del huesped. Filtros por tipo de habitacion y fecha
- Nueva Reserva: formulario con busqueda de huesped por documento, selector de fechas con verificacion de disponibilidad en tiempo real, selector de tipo de habitacion con tarifa calculada automaticamente
- Check-In: busqueda de reserva por codigo o nombre, confirmacion de datos del huesped, asignacion de habitacion disponible. Boton de confirmar check-in
- Folio del Huesped: vista de cuenta corriente con todos los cargos de la estancia (habitacion, restaurante, lavanderia, etc.). Total con IGV. Botones para agregar cargo y realizar checkout
- Housekeeping: lista de habitaciones por limpiar (checkout realizado hoy), con boton para marcar como LISTA. Filtro por piso
- Reportes de Ocupacion: tasa de ocupacion del dia y de la semana. Desglose por tipo de habitacion en barras. Revenue por tipo de habitacion
- Panel de Reservas: lista de reservas del dia (llegadas, en casa, salidas) con estado visual y accesos rapidos a check-in y checkout

Reglas de Negocio Críticas

- No se puede hacer check-in en habitación en MANTENIMIENTO o LIMPIEZA
- El precio se calcula con la tarifa vigente para las fechas solicitadas
- No se puede hacer checkout con cargos sin pagar en el folio
- Una habitación no puede tener dos reservas activas con fechas solapadas
- Al hacer checkout la habitación pasa automáticamente a LIMPIEZA

Criterios de aceptación

31. Plano del hotel: colores por estado actualizados en tiempo real
32. Reserva con cálculo de tarifa vigente funcionando end-to-end
33. Check-in y checkout con folio consolidado y validación de deuda
34. Housekeeping: flujo CHECKOUT → LIMPIEZA → DISPONIBLE visible en plano
35. Reportes: tasa de ocupación y revenue por tipo de habitación

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades, constraint de no-solapamiento de reservas, tarifas por temporada	Entidades principales sin constraint de solapamiento	Modelos basicos sin tarifas dinamicas	Modelos incompletos
API REST 15%	15%	Disponibilidad, reserva, check-in, checkout y folio con todas las validaciones	Flujo principal con tarifas o folio parciales	CRUD basico sin logica de disponibilidad por fechas	Menos del 50%
Logica tarifaria y folio 15%	15%	Tarifa vigente por temporada, folio con todos los cargos, bloqueo de checkout con deuda	Tarifa sin manejo de temporadas o folio sin bloqueo por deuda	Precio fijo sin logica tarifaria	Sin logica de tarifas ni folio
Frontend Web 20%	20%	Plano visual de habitaciones, calendario de reservas, flujo check-in/out y folio funcionando	Plano y check-in/out sin calendario de reservas ni folio visual	Solo formularios sin plano visual ni calendario	Frontend inexistente o desconectado
Integracion API-Frontend 10%	10%	Plano actualiza colores en tiempo real, tarifa se calcula dinamicamente al elegir fechas, folio muestra cargos al instante	Flujo integrado sin actualizacion dinamica del plano	Llamadas funcionan sin feedback visual de disponibilidad	Sin integracion

Auth y Permisos 10%	10%	JWT con roles (repcionista, housekeeping, admin), acceso diferenciado por rol	JWT con roles parciales	Autenticacion basica	Sin autenticacion
Testing 10%	10%	Tests de disponibilidad, solapamiento, check-in/out y folio. >= 60%	Tests del flujo principal 40-60%	Tests basicos < 40%	Sin tests
Docs y calidad 5%	5%	Swagger, README, sin N+1 en reservas con huesped y habitacion	Swagger y README basico	Solo README	Sin documentacion

PROYECTO 8

Gestion para Restaurantes y Food Service

Descripcion del Sistema

Plataforma para gestionar mesas, pedidos en sala, módulo de cocina (KDS), caja e inventario de insumos. Opera en tiempo real, con latencia mínima desde el pedido hasta la cocina.

Modelos de Datos Requeridos

Entidades:

- Mesa: numero, capacidad, zona, estado(LIBRE/OCUPADA/RESERVADA/LIMPIEZA)
- Categoria: nombre, es_bebida, orden_display
- Plato: nombre, descripcion, precio, categoria, tiempo_preparacion_min, disponible
- Insumo: nombre, unidad, stock_actual, stock_minimo, costo_unitario
- RecetaInsumo: plato, insumo, cantidad_por_porcion
- Comanda: mesa, mozo, estado(ABIERTA/EN_PREPARACION/LISTA/COBRADA/ANULADA), fecha_apertura
- LineaComanda: comanda, plato, cantidad, observacion, estado(PENDIENTE/EN_PREP/LISTO/ENTREGADO)
- Caja: turno, cajero, saldo_inicial, estado, fecha_apertura
- Pago: comanda, metodo, monto, vuelto, referencia

Endpoints Minimos Requeridos

- POST /api/comandas/ — Abrir comanda para una mesa
- POST /api/comandas/{id}/platos/ — Agregar platos verificando insumos
- POST /api/lineas/{id}/enviar-cocina/ — Enviar linea al KDS
- PATCH /api/lineas/{id}/estado/ — Cocina actualiza estado EN_PREP → LISTO
- GET /api/cocina/pendientes/ — Vista KDS: líneas pendientes y en preparación

Material didáctico de apoyo al estudiante

- POST /api/comandas/{id}/pagar/ — Cobrar, liberar mesa, descontar insumos
- GET /api/mesas/estado-actual/ — Estado de todas las mesas en tiempo real
- GET /api/reportes/ventas-turno/ — Resumen del turno: ventas, top platos

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Plano de Mesas (Salon): grid o layout visual con cada mesa coloreada segun estado (verde=libre, rojo=ocupada, amarillo=reservada, gris=limpieza). Click en mesa ocupada abre la comanda activa
- Tomar Pedido (Comanda): Catálogo de platos organizado por categoria con foto y precio. Click para agregar al pedido. Lista de items de la comanda con cantidades editables y campo de observacion por item
- Vista de Cocina (KDS — Kitchen Display System): pantalla dedicada para el cocinero con tarjetas de comandas activas. Cada tarjeta muestra: numero de mesa, platos con su estado y tiempo transcurrido. Boton para marcar cada plato como LISTO. Alerta visual si supera el tiempo de preparacion estimado
- Caja — Cobrar Comanda: lista de comandas listas para cobrar (estado LISTA). Vista del ticket con items y total. Selector de metodo de pago, campo de monto recibido y calculo automatico de vuelto
- Apertura y Cierre de Turno: formulario de saldo inicial al abrir. Resumen de ventas del turno con total por metodo de pago al cerrar
- Gestion de Menu: CRUD de platos y categorias con imagen, precio y flag de disponible. Lista de insumos con stock actual y alerta de stock bajo
- Reportes del Turno: ventas totales del turno, top 5 platos mas pedidos, tiempo promedio de preparacion por plato, comparativa de turnos en grafico

Reglas de Negocio Críticas

- No se puede abrir una comanda en una mesa con comanda ABIERTA
- Al agregar un plato, verificar stock de insumos vía RecetaInsumo
- Solo el cocinero puede cambiar línea a LISTO desde el KDS
- Al cobrar, los insumos se descuentan atómicamente vía RecetaInsumo
- Si stock de insumo clave llega a 0, el plato se marca automáticamente no disponible

Criterios de aceptación

36. Plano de mesas: estados actualizados, apertura de comanda desde el plano
37. Tomar pedido: catálogo navegable, agregar ítems con observaciones
38. KDS: tarjetas de comandas en tiempo real con botón de marcar listo
39. Caja: cobro con descuento de insumos y cálculo de vuelto funcionando
40. Reportes: top platos y ventas del turno con gráficas

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades con RecetaInsumo para control de stock e índices en comanda activa	Entidades principales sin RecetaInsumo	Modelos básicos sin trazabilidad de insumos	Modelos incompletos
API REST 15%	15%	Flujo salón-cocina-caja completo con verificación de stock de insumos atómicamente	Flujo principal sin atomicidad en descuento de insumos	CRUD de comandas sin módulo de cocina	Menos del 50%
Módulo cocina KDS 15%	15%	KDS con tarjetas en tiempo real, alerta de tiempo, separación salón/cocina por rol	KDS implementado sin alertas de tiempo o sin separación de roles	Solo listado de platos sin flujo KDS	Sin módulo de cocina
Frontend Web 20%	20%	Plano de mesas, comanda interactiva, KDS con tarjetas dinámicas y caja funcionando	Plano y comanda sin KDS en tiempo real o caja sin cálculo de vuelto	Formularios básicos sin plano visual ni KDS	Frontend inexistente o desconectado
Integración API-Frontend 10%	10%	KDS muestra nuevas líneas sin recargar, plano actualiza estados, stock de insumos visible en menú	Flujo integrado sin actualización automática del KDS ni del plano	Llamadas funcionan sin feedback en tiempo real	Sin integración
Auth y Permisos 10%	10%	JWT con roles (mozo, cocinero, cajero, admin), vistas diferenciadas por rol	JWT con roles parciales	Autenticación básica	Sin autenticación
Testing 10%	10%	Tests de comanda, cocina, cobro y descuento de insumos. $\geq 60\%$	Tests del flujo principal 40-60%	Tests básicos $< 40\%$	Sin tests
Docs y calidad 5%	5%	Swagger, README, sin N+1 en comandas con líneas	Swagger y README básico	Solo README	Sin documentación

PROYECTO 9

Control Nutricional y Tracking de Hábitos

Descripción del Sistema

Plataforma web y móvil para registrar la alimentación diaria, hacer seguimiento de hábitos saludables, visualizar el progreso mediante gráficas y recibir recomendaciones según las metas personales.

Modelos de Datos Requeridos

Entidades:

- PerfilUsuario: usuario, peso_kg, talla_cm, edad, sexo, nivel_actividad, objetivo
- MetaNutricional: perfil, calorías_meta, proteínas_g, carbohidratos_g, grasas_g, agua_ml
- Alimento: nombre, marca, calorías_100g, proteínas_100g, carbohidratos_100g, grasas_100g, es_peruano
- RegistroComida: usuario, fecha, tipo_comida(DESAYUNO/ALMUERZO/CENA/SNACK)
- ItemRegistro: registro, alimento, cantidad_g, calorías_total, proteínas_total, carbohidratos_total, grasas_total
- RegistroHbito: usuario, fecha, vasos_agua, horas_sueno, pasos, minutos_ejercicio, tipo_ejercicio
- Logro: nombre, descripcion, tipo, icono, condicion (JSON)
- LogroUsuario: usuario, logro, fecha_obtenido

Endpoints Minimos Requeridos

- POST /api/registros-comida/ — Registrar comida con cálculo automático de macros
- GET /api/alimentos/buscar/?q= — Búsqueda por nombre (incluye alimentos peruanos)
- GET /api/dashboard/hoy/ — Calorías consumidas vs meta, macros, hábitos del día
- POST /api/habitos/ — Registrar hábitos del día (agua, sueño, ejercicio)
- GET /api/progreso/semanal/ — Calorías, macros y hábitos de los últimos 7 días
- GET /api/progreso/mensual/?mes= — Resumen mensual con tendencias
- POST /api/perfil/metadatos/ — Calcular y guardar metas vía fórmula Harris-Benedict
- GET /api/logros/ — Logros obtenidos y en progreso del usuario

Vistas Web Requeridas

Vistas / Pantallas Requeridas (Frontend Web)

Stack aceptado: Django Templates + Bootstrap 5 | React/Vue + API REST

- Dashboard Diario: anillo circular de progreso de calorías (consumido vs meta). Barras de macros (proteínas, carbohidratos, grasas). Checklist de hábitos del día (vasos de agua, sueño, ejercicio). Resumen de comidas del día con total por tipo
- Agregar Comida: buscador con autocompletado de alimentos. Al seleccionar un alimento, mostrar tabla nutricional por 100g. Campo de cantidad con calculo automatico de macros en tiempo real. Selector de tipo de comida (Desayuno, Almuerzo, etc.)
- Tracker de Hábitos: formulario simple con inputs numericos para vasos de agua y horas de sueño, contador de pasos y minutos de ejercicio con selector de tipo. Historico de los ultimos 7 dias en tabla
- Progreso Semanal: grafico de lineas con calorías diarias vs meta (7 días). Grafico de barras apiladas de macros por día. Porcentaje de días en que se cumplieron los hábitos
- Progreso Mensual: grafico de tendencia de peso si el usuario lo registra. Promedio diario de calorías del mes. Días de cumplimiento de meta nutricional
- Perfil y Metas: formulario de datos personales (peso, talla, edad, nivel de actividad, objetivo). Al guardar, mostrar las metas calculadas automaticamente via Harris-Benedict con opcion de ajustar manualmente
- Logros: galeria de badges (iconos) clasificados en obtenidos (color) y bloqueados (gris). Click en logro bloqueado muestra el progreso actual hacia el criterio de desbloqueo (ej: "5/7 días consecutivos registrando")

Reglas de Negocio Críticas

- Calorías y macros del ItemRegistro se calculan automáticamente (cantidad * nutrientes / 100)
- La meta calórica se calcula con la fórmula Harris-Benedict según el perfil
- Solo un registro hábito por usuario por día
- Si se supera el 110% de la meta calórica, mostrar alerta en el dashboard
- Los logros se verifican automáticamente al guardar registros o hábitos

Criterios de aceptación

41. Dashboard diario: anillo de calorías y barras de macros en tiempo real al agregar comidas
42. Buscador de alimentos: autocompletado con cálculos dinámicos al cambiar la cantidad
43. Tracker de hábitos: registro diario con validación de uno por día
44. Progreso: gráficas semanales y mensuales con datos reales del usuario
45. Logros: desbloqueo automático con feedback visual inmediato

Rúbrica de Evaluación

Área / Módulo	Peso	Logrado (100%)	En Progreso (70%)	Básico (40%)	Insuficiente (0%)
Modelos y DB 15%	15%	Todas las entidades, calculo de macros automatico, unique en habito por dia	Entidades principales sin unique o con calculo manual	Modelos basicos sin formula Harris-Benedict	Modelos incompletos
API REST 15%	15%	Registro, dashboard, progreso y logros con calculos correctos	Registro y dashboard, progreso o logros parciales	CRUD basico sin calculos automaticos de macros	Menos del 50%
Logica nutricional 15%	15%	Harris-Benedict correcto, macros calculados automaticamente, alertas de exceso calórico	Calculo de macros correcto pero formula incorrecta o sin alertas	Calculo basico sin formula personalizada	Sin logica nutricional
Frontend Web 20%	20%	Dashboard con anillo y barras animadas, buscador con calculo dinamico, galeria de logros y graficas de progreso	Dashboard y buscador sin animaciones o sin graficas de progreso	Solo formularios sin dashboard visual ni graficas	Frontend inexistente o desconectado
Integracion API-Frontend 10%	10%	Macros calculan en browser al cambiar cantidad, dashboard actualiza sin recargar al agregar comida, logros se desbloquean con feedback inmediato	Flujo integrado sin actualizacion dinamica del dashboard	Llamadas funcionan sin feedback de progreso	Sin integracion

Auth y Permisos 10%	10%	JWT, cada usuario solo accede a sus datos, perfil y metas personalizados	JWT con isolation de datos parcial	Autenticacion basica	Sin autenticacion
Testing 10%	10%	Tests de calculo de macros, Harris-Benedict, dashboard y logros. >= 60%	Tests de registro y dashboard 40-60%	Tests basicos < 40%	Sin tests
Docs y calidad 5%	5%	Swagger, README con formulas nutricionales documentadas, sin N+1	Swagger y README basico	Solo README	Sin documentacion

IV. Consigna

Cada producto debe tener lo siguiente:

1. Frontend usando estilos css, se pueden usar algunas librerías tipo Bootstrap o Tailwind
2. Se debe hacer haciendo uso del patrón modelo vista controlador
3. Cada solución debe implementar un API rest
4. Se debe aplicar el principio de separación de capas (modelos, vistas, formularios).
5. Toda entrada de datos debe ser validada correctamente.
6. Debe utilizar control de versiones (Git) y estar alojado en un repositorio privado o público y dar acceso al docente del curso.
7. La documentación debe incluir: descripción técnica, modelo de datos, casos de uso, instrucciones para
8. ejecutar.
9. El equipo debe presentar el proyecto con una demo funcional.

V. Estructura básica del producto académico

Para la presentación del Informe académico se tendrá en cuenta la siguiente estructura:

1. **Carátula**
 - Logotipo oficial y actual de la USS
 - Nombre de la facultad y escuela
 - Título del Informe: <<Nombre del proyecto>>
 - Integrantes: Apellidos y nombres de los estudiantes
 - Curso, Docente, semestre

El proyecto debe tener la siguiente estructura dentro del código fuente (*vamos a tomar como ejemplo un Sistema Web para la Gestión de Tesorería*):

2. **Diseño del modelo de datos**

Aplicar principios *matemáticos y de ciencias de la computación* para representar correctamente las relaciones entre entidades del sistema y garantizar la **consistencia** y

Indicaciones:

- Identifica las **entidades principales** (Ingresos, Egresos, Categorías, Proveedores, Usuarios).
- Define las **relaciones y claves primarias/foráneas**.
- Asegura la integridad referencial (por ejemplo, un ingreso debe pertenecer a una categoría y un usuario).
- Usa **notación estándar (MER o UML)** para representar el modelo.
- Implementa el modelo en el ORM de Django, Flask o framework asignado.

Evidencias:

- Diagrama entidad-relación o UML con descripciones de campos.
- Código de modelos implementado en el ORM.
- Captura de base de datos generada automáticamente por migraciones.
- Breve reflexión: “El modelo permite representar la realidad financiera de manera estructurada y medible.”

3. Lógica de negocio y validaciones

Implementar procesos lógicos, algoritmos y modelos matemáticos que aseguren el funcionamiento correcto en base a los requerimientos del sistema por ejemplo (Pronóstico de flujo de caja, Optimización de caja y pagos, Simulación de liquidez), aplicando principios computacionales y buenas prácticas de ingeniería. En este apartado se debe asegurar el uso de modelos matemáticos avanzados, aplicados y programados para implementar las funcionalidades del sistema. En este ejemplo el sistema está relacionado con la administración de recursos financieros (tesorería), así como aspectos económicos o financieros que son disciplinas externas a nuestra especialidad por lo tanto se requiere revisarlas, comprenderlas y aplicarlas, así como utilizar los modelos matemáticos avanzados de estas disciplinas que permitan programar las funcionalidades del sistema.

Instrucciones

- Desarrolla funciones que gestionen el registro y actualización de movimientos (Pronóstico de flujo de caja, Optimización de caja y pagos, Simulación de liquidez)
- Definir los conceptos de otras disciplinas relacionadas con el sistema en desarrollo
- Deberá explicar detalladamente el modelo matemático seleccionado paso por paso considerando las ecuaciones y su explicación contextualizada con el problema que está resolviendo, además deberá agregar el código que implementa estos modelos matemáticos.
- Implementa los modelos matemáticos (Pronóstico de flujo de caja, Optimización de caja y pagos, Simulación de liquidez, etc.)

4. Interfaz y experiencia de usuario

Diseñar una interfaz intuitiva, accesible y coherente con los principios de **usabilidad, inclusión digital y sostenibilidad**.

Instrucciones

- Crea formularios de ingreso de datos con validación visual (campos obligatorios, formatos).
- Aplica **Bootstrap, Tailwind o CSS modular** para mantener consistencia visual.
- Implementa menús o paneles que permitan visualizar ingresos, egresos y reportes.
- Garantiza una experiencia fluida tanto en escritorio como en dispositivos móviles.

Ejemplo

“El formulario de registro de movimientos permite ingresar tipo, fecha, monto y categoría.

Si el usuario ingresa un valor negativo, el sistema muestra una alerta: ‘El monto no puede ser menor a cero’.”

Evidencias

- Capturas de pantalla de formularios y paneles.
- Descripción de la experiencia del usuario final.
- Breve reflexión: “La interfaz facilita el uso responsable del sistema y contribuye a una gestión sostenible sin uso de papel.”

5. Reportes y consultas

Aplicar principios matemáticos y analíticos para **interpretar la información financiera** del sistema y **tomar decisiones basadas en datos**.

Instrucciones

- Genera reportes de flujo de caja por rango de fechas.
- Incluye gráficos de barras o líneas (por ejemplo, ingresos vs. egresos).
- Implementa filtros dinámicos (por fecha, categoría o tipo de movimiento).
- Calcula indicadores clave: saldo promedio, tasa de variación mensual, proporción de egresos sobre ingresos.

Ejemplo de tabla de reporte

Fecha	Tipo	Categoría	Monto (S/.)	Saldo acumulado
2025-09-01	Ingreso	Ventas	4,500.00	4,500.00
2025-09-03	Egreso	Compras	1,800.00	2,700.00
2025-09-05	Egreso	Servicios	500.00	2,200.00

Evidencias:

- Capturas de los reportes y sus gráficos.
- Análisis interpretativo: “Se observa un incremento del 15% en ingresos respecto al mes anterior.”
- Cálculo de indicadores y justificación de su relevancia.
- Reflexión sobre la **sostenibilidad económica y tecnológica** del sistema.

6. Análisis interdisciplinario y sostenibilidad del sistema

Aplicar principios de **otras disciplinas relevantes** (administración, contabilidad, gestión financiera, sostenibilidad organizacional y medioambiental) para **analizar y fundamentar la pertinencia del sistema web**, considerando su impacto en la eficiencia, transparencia y desarrollo sostenible de la organización.

Instrucciones

- **Relaciona** el funcionamiento del sistema con **procesos administrativos y contables reales**, explicando cómo contribuye a una mejor gestión del dinero, recursos y decisiones.
- **Analiza el impacto social y ambiental** del sistema, por ejemplo: reducción del uso de papel, ahorro de tiempo, digitalización segura y transparente. Cuantifique el impacto obtenido.
- **Justifica la sostenibilidad** de la solución en términos de escalabilidad, mantenimiento y beneficio a largo plazo.

Evidencias sugeridas

- Cuadro comparativo antes/después del sistema, mostrando mejoras en tiempos, errores o uso de materiales.
- Cálculo de impacto económico y sostenible (porcentaje de reducción de costos o recursos).

Ejemplo

“El sistema de gestión de tesorería no solo mejora los procesos de control financiero, sino que también contribuye al desarrollo sostenible. Al digitalizar los reportes y eliminar el uso de registros en papel, la empresa reduce su huella ecológica en un 40%. Además, la automatización del flujo de caja permite una gestión más eficiente de recursos económicos y humanos, mejorando la transparencia y la toma de decisiones.

Desde la perspectiva administrativa y contable, el sistema fortalece la trazabilidad y reduce los errores humanos asociados al manejo manual del efectivo.....”

Ejemplo de tabla antes vs después

KPI	Antes (manual)	Después (sistema)	Diferencia	Mejora %
Tiempo de registro por movimiento (min)	3.5	1.2	-2.3	-65.7%
Tiempo de conciliación diaria (min)	45	15	-30	-66.7%
Errores de digitación por mes (qty)	28	6	-22	-78.6%
Reprocesos por errores (horas/mes)	18	4	-14	-77.8%

7. Anexos: obligatoriamente incluye lo siguiente

- Se debe anexas una rubrica de evaluación del atributo del graduado: “[AG-C08] Análisis de Problemas”, por cada miembro de equipo. Puedes encontrar el INSTRUMENTO DE EVALUACIÓN DE LOS ATRIBUTOS DEL GRADUADO, en el anexo 2 de este documento.

VI. Características formales del trabajo

- El trabajo deberá ser entregado a través del Aula virtual opción: **Actividades Obligatorias**, en: **EVIDENCIA DE PRODCUTO (EP): Proyecto de desarrollo de una solución web empresarial**
- El trabajo debe incluir los archivos adjuntos del trabajo en equipo con tu grupo.

VII. Insumos o recursos tecnológicos a emplear

Para realizar esta actividad necesariamente debes utilizar los siguientes recursos y herramientas:

- Material de estudio sobre el curso

VIII. Criterios e indicadores de evaluación

Para la evaluación del trabajo, se tendrá en cuenta los siguientes criterios:

El puntaje detallado de cada uno de estos criterios se encuentra en la “**Lista de Contejo**” por lo tanto se recomienda leer detenidamente dicho documento, que se ubica registrado en el aula virtual.

IMPORTANTE

Si se evidencia que el trabajo académico presentado ha sido copiado y pegado de internet o de otra fuente, sin el debido procesamiento de la información y sin las adecuadas citas y referencias de la fuente, automáticamente se invalida el trabajo (con nota desaprobatoria)

ANEXO 01:**Alineación de los atributos del graduado con secciones de la guía**

Indicador del Atributo [AG-C08]	Sección / Punto de la Guía (Enumerado)	Descripción resumida	Justificación de la alineación
AG-C08-C1 Principios de matemáticas	5.1 Diseño del modelo de datos	Se definen entidades, relaciones y claves, garantizando consistencia y trazabilidad.	Aplica principios matemáticos para representar y analizar relaciones entre datos del sistema.
AG-C08-C1	5.4 Reportes y consultas	Se generan indicadores financieros, gráficos y cálculos de variación.	Usa fundamentos matemáticos para analizar resultados y medir desempeño.
AG-C08-C1	5.4 Reportes y consultas – Reflexión sobre sostenibilidad	Evalúa resultados e impacto económico del sistema.	Emplea cálculos matemáticos para valorar eficiencia y sostenibilidad del sistema.
AG-C08-C2 Principios de ciencias de la computación	5.2 Lógica de negocio y validaciones	Implementa algoritmos y modelos computacionales según los requerimientos.	Aplica fundamentos de ciencias de la computación para resolver problemas

			mediante programación.
AG-C08-C2	5.2 Lógica de negocio – Modelos matemáticos aplicados	Explica y programa modelos matemáticos utilizados en el sistema.	Analiza y justifica los modelos con base en razonamiento lógico y computacional.
AG-C08-C2	5.4 Reportes y consultas – Análisis interpretativo	Interpreta resultados mediante técnicas y herramientas analíticas.	Aplica principios científicos para analizar datos y validar resultados.
AG-C08-C3 Principios de otras disciplinas relevantes	5.3 Interfaz y experiencia de usuario	Diseña una interfaz accesible, usable y sostenible.	Integra conocimientos de diseño y usabilidad para resolver problemas centrados en el usuario.
AG-C08-C3	5.5 Análisis interdisciplinario y sostenibilidad del sistema	Analiza impactos económicos, administrativos y ambientales.	Aplica principios de administración y sostenibilidad en la evaluación del sistema.
AG-C08-C3	5.5 Análisis interdisciplinario – Cálculo de impacto y mejora	Cuantifica mejoras antes y después del sistema.	Usa indicadores de eficiencia e impacto sustentados en disciplinas externas.

INSTRUMENTO DE EVALUACIÓN DE LOS ATRIBUTOS DEL GRADUADO

CURSO: TALLER DE LENGUAJES DE PROGRAMACIÓN / CICLO: IX

Apellidos y nombres el estudiante:

Descripción: A continuación, se detalla la rúbrica de evaluación del atributo del graduado del programa de Ingeniería de Sistemas.

[AG-C08] Análisis de Problemas

Identifica, formula, busca información y analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios de matemáticas, ciencias de la computación y otras disciplinas relevantes, con las consideraciones necesarias para el desarrollo sostenible.

ID	Indicadores de desempeño	Peso	Niveles de logro			
			(1) Inicial	(2) En desarrollo	(3) Logrado	(4) Avanzado
AG-C 08-C1	Identifica, formula, busca información y analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios de matemáticas, con las consideraciones necesarias para el desarrollo sostenible.	33,34%	Identifica problemas complejos de computación llegando a conclusiones fundamentadas usando principios de matemáticas, con las consideraciones necesarias para el desarrollo sostenible.	Formula complejos de computación llegando a conclusiones fundamentadas usando principios de matemáticas, con las consideraciones necesarias para el desarrollo sostenible.	Busca información de problemas complejos de computación llegando a conclusiones fundamentadas usando principios de matemáticas, con las consideraciones necesarias para el desarrollo sostenible.	Analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios de matemáticas, con las consideraciones necesarias para el desarrollo sostenible.
AG-C 08-C2	Identifica, formula, busca información y analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios ciencias de la computación, con las consideraciones necesarias para el desarrollo sostenible.	33,33%	Identifica problemas complejos de computación llegando a conclusiones fundamentadas usando principios ciencias de la computación, con las consideraciones necesarias para el desarrollo sostenible.	Formula problemas complejos de computación llegando a conclusiones fundamentadas usando principios ciencias de la computación, con las consideraciones necesarias para el desarrollo sostenible.	Busca información de problemas complejos de computación llegando a conclusiones fundamentadas usando principios ciencias de la computación, con las consideraciones necesarias para el desarrollo sostenible.	Analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios ciencias de la computación, con las consideraciones necesarias para el desarrollo sostenible.
AG-C 08-C3	Identifica, formula, busca información y analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios de otras disciplinas relevantes, con las consideraciones necesarias para el desarrollo sostenible.	33,33%	Identifica problemas complejos de computación llegando a conclusiones fundamentadas usando principios de otras disciplinas relevantes, con las consideraciones necesarias para el desarrollo sostenible.	Formula problemas complejos de computación llegando a conclusiones fundamentadas usando principios de otras disciplinas relevantes, con las consideraciones necesarias para el desarrollo sostenible.	Busca información de problemas complejos de computación llegando a conclusiones fundamentadas usando principios de otras disciplinas relevantes, con las consideraciones necesarias para el desarrollo sostenible.	Analiza problemas complejos de computación llegando a conclusiones fundamentadas usando principios de otras disciplinas relevantes, con las consideraciones necesarias para el desarrollo sostenible.

_____ (firma)

Docente:.....

DNI: